Sree — Task Checklist & Antigravity Prompts

Udaan AI · AI/LLM layer + Job Seekers + Senior Citizens

How to use: paste `Antigravity_Prompt_Sree.md` once as project context. Then run the prompts below in order. Tick each box when its acceptance check passes.

✅ MY CHECKLIST (mark done one by one)

- ☐ **T0** — Confirm the schema/field names with Puja (the data contract)
- ☐ **T1** — Set up my folders + Gemini env
- ☐ **T2** — Profile Agent (parse text → profile + typed fallback)
- ☐ **T3** — Explanation + multilingual layer
- ☐ **T4** — Scam Shield (rule-based)
- ☐ **T5** — Skill → free-scheme map
- ☐ **T6** — Missed-Opportunity Detector + Alert Center
- ☐ **T7** — Job Seekers data pack (verified)
- ☐ **T8** — Senior Citizens data pack (verified)
- ☐ **T9** — Tests for all my functions
- ☐ **T10** — Integration: profile → engine → plain reasons (end-to-end)
- ☐ **T11** — Wire voice input + language toggle (with Ujasvi)
- ☐ **T12** — Rehearse the regret → relief demo beat

🤖 AGENT PROMPTS (run in order)

T1 — Setup

In the backend folder, create these directories if missing: integrations/, logic/, data/, tests/.

Add google-generativeai and pydantic to requirements. Create a .env.example with LLM_API_KEY=.

Create integrations/llm.py that loads LLM_API_KEY from .env and exposes a single helper

call_gemini(prompt: str) -> str that returns the model's text, and returns None on any error

(so callers can fall back). Do not put any business logic here.

Done when: `call_gemini` works with a key and returns None without one.

T2 — Profile Agent

Create integrations/profile_agent.py with:

- parse_profile(text, module, lang="en") -> dict | None : calls call_gemini with the extraction

  prompt, parses strict JSON into the Profile shape from the data contract, validates types with

  pydantic; returns None on any failure.

- profile_from_form(form, module) -> dict : fills the SAME Profile shape from a typed form (fallback).

Use this extraction prompt:

"You extract a user profile as JSON. Output ONLY valid JSON with keys: name, age (int), gender,

category (general/OBC/SC/ST/EWS/PwD/women), state, qualification, income (int annual rupees),

skills (array), interests (array). Use null if unknown. User (language={lang}) said: {text}"

Done when: a sample sentence returns a correct Profile dict; bad input returns None.

T3 — Explanation layer

Create integrations/explain.py with explain(engine_result, language="en") -> dict.

Take EngineResult (verdict, reasons[], missing_documents, missing_skills, readiness) and ask Gemini

to rewrite each reason as ONE simple sentence in the given language (te/hi/en), keeping meaning

EXACTLY. Return {verdict, reasons_plain[], missing_documents, missing_skills, readiness}.

The verdict and all numbers must pass through unchanged. If call_gemini returns None, return the

original reasons unchanged. The LLM must never change the verdict.

Done when: verdict/numbers unchanged; reasons come back in the chosen language.

T4 — Scam Shield

Create logic/scam_shield.py with check_scam(text) -> {is_scam, flags}. Rule-based, NO LLM.

Flag if the posting: asks for money/fee/payment; asks for bank/UPI/OTP/card details; claims a

government job from a non-.gov.in/.nic.in domain; says "guaranteed government job"; uses urgency

("apply within X hours"). is_scam = len(flags) >= 1. Add a few unit-test examples in a docstring.

Done when: "Pay ₹5000 to confirm your govt job" → is_scam True with flags.

T5 — Skill → free-scheme map

Create logic/skill_to_scheme.py with a dict SKILL_TO_SCHEME mapping a missing skill to a free

government scheme as (name, url): e.g. typing -> ("PMKVY typing/CSC certification",

"https://pmkvyofficial.org"); quantitative_aptitude, spoken_english, open_source, computer_basics.

Add free_fix(missing_skills) -> list of {skill, scheme, url} for the skills that have a mapping.

Done when: free_fix(["typing"]) returns the PMKVY entry.

T6 — Missed + Alerts

Create logic/missed_and_alerts.py with:

- missed(profile, opportunities, check_eligibility) -> list : opportunities where is_active is False

  AND check_eligibility(profile, opp)['verdict'] == 'eligible'. These are the regret hook.

- alerts(profile, opportunities, within_days=30) -> list : active opportunities whose close_date is

  within within_days from today, each as {title, close_date, message}. In-app only, no Telegram.

Done when: a past matched opportunity shows in missed(); a soon-closing one shows in alerts().

T7 — Job Seekers data pack

Create data/jobseekers.csv with columns exactly matching the Opportunity fields (JSON columns as

JSON strings). Add 6 rows: SSC CGL, SSC CHSL, RRB NTPC (Graduate), TSPSC Group 4 (Telangana

domicile), one NAPS apprenticeship (with stipend), and PMKVY (skilling/free-fix).

Use standard central age relaxation (OBC +3, SC/ST +5, PwD +10) but ADD a TODO comment per row to

verify the exact age limits and relaxation against the official site listed in the source column.

Mark one row is_active=false (a past deadline) so the Missed Detector has data.

Done when: CSV loads via Puja's loader without errors. (Then verify numbers manually.)

T8 — Senior Citizens data pack

Create data/senior_citizens.csv with the same columns. Add 5 rows: IGNOAPS old-age pension (age 60+,

BPL), Ayushman Bharat PM-JAY (₹5 lakh health), Rashtriya Vayoshri Yojana (senior, BPL), Senior

Citizen Savings Scheme (age 60+), and a Telangana state old-age pension. Add a TODO per row to verify

amounts/eligibility against the official source. Mark one is_active=false for the Missed Detector.

Done when: CSV loads without errors. (Then verify manually.)

T9 — Tests

Create tests/test_sree.py with pytest tests:

- check_scam flags a paid-job posting and passes a clean one.

- parse_profile returns the right keys on a mocked LLM response and None on bad JSON.

- explain keeps verdict and readiness unchanged.

- free_fix returns the right scheme for a known skill.

Run pytest and fix until green.

Done when: all tests pass.

T10 — Integration (manual + agent)

Write a small script scripts/demo_flow.py that: takes a sample sentence -> parse_profile ->

loads jobseekers.csv -> runs Puja's check_eligibility over it -> picks one eligible opportunity ->

runs explain() in Telugu and prints the plain-language reasons, readiness, missing docs, and any

missed opportunities. This is the end-to-end proof of my layer.

Done when: the script prints a correct profile, eligibility reasons in Telugu, and a missed item.

T11 — Voice + language (with Ujasvi)

(Coordinate with Ujasvi.) The frontend mic (Web Speech API) sends a transcript string to a backend

endpoint that calls parse_profile(text, module, lang). Confirm the language toggle passes te/hi/en

through to explain(). No backend voice processing needed — the browser does speech-to-text.

Done when: speaking a sentence fills the profile and explanations render in the chosen language.

T12 — Rehearse the wow

(No code.) Rehearse the Job Seekers beat: profile by voice -> "you missed ₹X" regret ->

eligibility reasoning (OBC +3 -> 33) -> readiness 65% -> free PMKVY fix -> scam shield flags a fake

posting. Time it to ~3 minutes. Prepare the "why not ChatGPT" answer.

Done when: you can run the beat smoothly end-to-end from memory.

Dependency reminder: T2–T9 you can build now in parallel. T10 needs Puja's `check_eligibility` (import her function). T11 needs Ujasvi's mic. T7/T8 numbers must be verified before the demo.